Exercise 3: Bronze to Silver with Parquet

Objective

Add a silver layer task that reads JSON from bronze and saves it as Parquet in silver. Then add a second data source running in parallel.

Prerequisites

- Exercise 2 completed

About the Medallion Architecture

This exercise implements the **bronze/silver** pattern, also known as **raw/staging/core** in dbt and traditional data warehousing:

Medallion	dbt/Warehouse	What it does
Bronze	**Raw/Landing**	Raw data as-is from the source
Silver	**Staging**	Cleaned, typed, deduplicated
Gold	**Core/Marts**	Business models, aggregations

In this exercise we go from bronze (JSON) to silver (Parquet). The homework assignment takes it further to gold.

Step 1: Review the Silver Layer DAG

Open the file `dags/exercise3_silver.py` and study the code:

```
import json
import os
from datetime import datetime

import boto3
import pandas as pd
import requests
import s3fs
from airflow import DAG
from airflow.decorators import task

BRONZE_BUCKET = "bronze"
SILVER_BUCKET = "silver"
S3_ENDPOINT = os.environ.get("AWS_ENDPOINT_URL")

with DAG(
    dag_id="exercise3_silver",
    start_date=datetime(2026, 1, 1),
    schedule="0 14 * * *",
    catchup=False,
    default_args={
```

```
        "retries": 1,
    },
) as dag:

    @task()
    def fetch_api():
        response = requests.get("https://api.chucknorris.io/jokes/search?
query=computer")
        data = response.json()

        local_path = "/tmp/chuck_jokes.json"
        with open(local_path, "w") as f:
            json.dump(data["result"], f)

        return local_path

    @task()
    def upload_to_s3(local_path, ds=None):
        s3 = boto3.client("s3", endpoint_url=S3_ENDPOINT)
        s3_key = f"chuck_jokes/_created_date={ds}/dump.json"
        s3.upload_file(local_path, BRONZE_BUCKET, s3_key)

        return s3_key

    @task()
    def bronze_to_silver(s3_key, ds=None):
        fs = s3fs.S3FileSystem(
            endpoint_url=S3_ENDPOINT,
            key="minioadmin",
            secret="minioadmin",
        )

        with fs.open(f"{BRONZE_BUCKET}/{s3_key}") as f:
            df = pd.read_json(f)

        silver_key = f"{SILVER_BUCKET}/chuck_jokes/_created_date=
{ds}/data.snappy.parquet"
        with fs.open(silver_key, "wb") as f:
            df.to_parquet(f, compression="snappy", index=False)

        return f"s3://{silver_key}"

    local_file = fetch_api()
    s3_key = upload_to_s3(local_file)
    bronze_to_silver(s3_key)
```

Step 2: Run and Verify

1. Trigger `exercise3_silver` from the Airflow UI
2. Check all 3 tasks succeed

Verify in MinIO

Open http://localhost:9001 and check both buckets:

- **bronze** > `chuck_jokes/_created_date=.../dump.json` (JSON, ~10 KB)
- **silver** > `chuck_jokes/_created_date=.../data.snappy.parquet` (Parquet, ~11 KB)

The Parquet file is a compressed columnar format optimized for analytics.

Step 3: Review the Second Data Source

The DAG already includes a second API source with its own parallel pipeline:

```
    @task()
    def fetch_universities():
        response = requests.get("http://universities.hipolabs.com/search?
country=Spain")
        data = response.json()

        local_path = "/tmp/universities_spain.json"
        with open(local_path, "w") as f:
            json.dump(data, f)

        return local_path

    @task()
    def upload_universities_to_s3(local_path, ds=None):
        s3 = boto3.client("s3", endpoint_url=S3_ENDPOINT)
        s3_key = f"universities/_created_date={ds}/dump.json"
        s3.upload_file(local_path, BRONZE_BUCKET, s3_key)

        return s3_key

    @task()
    def universities_bronze_to_silver(s3_key, ds=None):
        fs = s3fs.S3FileSystem(
            endpoint_url=S3_ENDPOINT,
            key="minioadmin",
            secret="minioadmin",
        )

        with fs.open(f"{BRONZE_BUCKET}/{s3_key}") as f:
            df = pd.read_json(f)

        silver_key = f"{SILVER_BUCKET}/universities/_created_date=
{ds}/data.snappy.parquet"
        with fs.open(silver_key, "wb") as f:
            df.to_parquet(f, compression="snappy", index=False)

        return f"s3://{silver_key}"

    # Pipeline 1: Chuck Jokes
```

```
    local_file = fetch_api()
    s3_key = upload_to_s3(local_file)
    bronze_to_silver(s3_key)

    # Pipeline 2: Universities (runs in parallel)
    uni_file = fetch_universities()
    uni_s3_key = upload_universities_to_s3(uni_file)
    universities_bronze_to_silver(uni_s3_key)
```

Step 4: Verify Parallel Execution

1. Trigger the DAG
2. In the **Graph view**, confirm both pipelines run in parallel
3. Open http://localhost:9001 and verify both buckets have data for both sources:

```
bronze/
  chuck_jokes/_created_date=.../dump.json
  universities/_created_date=.../dump.json

silver/
  chuck_jokes/_created_date=.../data.snappy.parquet
  universities/_created_date=.../data.snappy.parquet
```

Questions

1. Why is Parquet better than JSON for the silver layer?
2. What does `compression="snappy"` do?
3. How does Airflow know the two pipelines can run in parallel?
4. What would the Graph view look like?